



Compiler Construction PROJECT REPORT

Submitted to

Ms. Ghazala

Submitted by

Muhammad Zaid 2022-CS-622

Muhammad Bilal 2022-CS-629

Muhammad Talha 2022-CS-630

Department of Computer Science, New Campus
University of Engineering and Technology, Lahore

Contents

1	Introduction	2
1.1	Objectives	2
2	Problem Statement	2
3	System Design	3
3.1	Architecture	3
3.2	Module Organization	3
4	Grammar Rules	4
4.1	Program Structure	4
4.2	Declarations	4
4.3	Functions	4
4.4	Statements	4
4.5	Control Structures	5
4.6	Expressions	5
4.7	Operator Precedence	6
5	Implementation Details	6
5.1	Lexical Analysis	6
5.2	Syntax Analysis	7
5.3	Symbol Table Management	7
5.4	Semantic Analysis	7
5.5	Error Handling	8
5.6	Intermediate Code Generation	9
6	Design Choices	9
6.1	Tool Selection	9
6.2	Data Structures.....	10
6.3	Error Recovery Strategy	10
7	Testing	10
7.1	Test Cases	10
7.2	Test Results.....	11
8	Conclusion	11

1 Introduction

This project implements a mini-compiler for a restricted subset of a C-like programming language. The compiler processes source code through all major compiler phases: **Lexical Analysis**, **Syntax Analysis**, **Semantic Analysis**, and **Intermediate Code Generation**. The implementation uses automated tools (Flex and Bison) to demonstrate system-level integration of compiler phases.

1.1 Objectives

- Implement a lexical analyzer (scanner) to tokenize input source code
- Develop a syntax analyzer (parser) to validate structure against grammar rules
- Create a semantic analyzer to handle type checking, variable declarations, and scope resolution
- Implement a symbol table manager and error handler
- Generate intermediate code representation
- Integrate error handling and error recovery techniques

2 Problem Statement

Design and implement a mini-compiler for a restricted subset of a C-like programming language that supports:

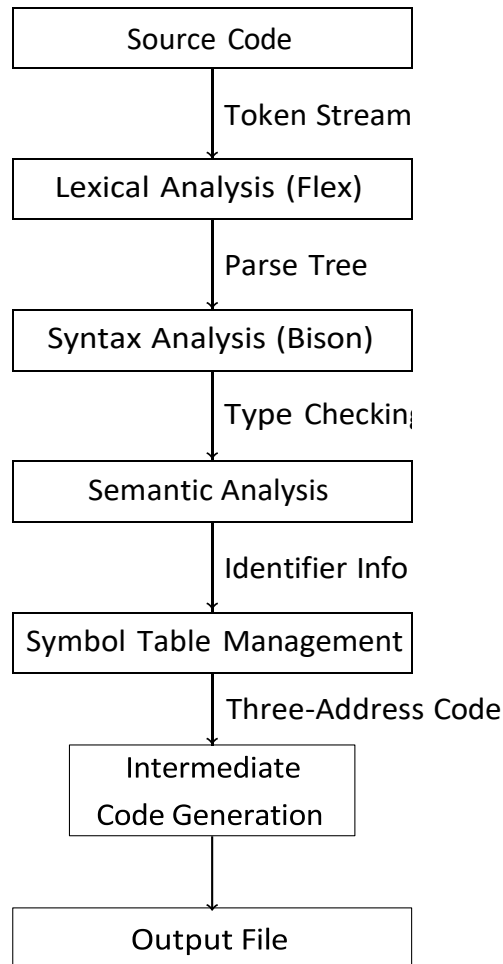
- **Data Types:** int, float, bool, void
- **Control Structures:** if-else, while, for
- **Functions:** Function definitions (without parameters)
- **Statements:** Declarations, assignments, input (read), output (write), return
- **Operators:** Arithmetic (+, -, *, /, %), Comparison (<, <=, >, >=, ==, !=), Logical (&&, ||, !)

The compiler must process source programs through lexical analysis, syntax analysis, semantic analysis, and intermediate code generation, with integrated error handling and symbol table management.

3 System Design

3.1 Architecture

The compiler follows a modular, phase-based architecture:



3.2 Module Organization

1. **Lexical Analyzer** (src/scanner.l) - Tokenizes input using Flex
2. **Syntax Analyzer** (src/parser.y) - Parses tokens using Bison
3. **Symbol Table** (src/symbol table.c) - Manages identifiers and scopes
4. **Error Handler** (src/error handler.c) - Collects and reports errors
5. **Semantic Analyzer** (src/semantic.c) - Performs type checking
6. **Intermediate Code Generator** (src/icode.c) - Generates three-address code

4 Grammar Rules

4.1 Program Structure

program $\rightarrow$ declaration* function*

4.2 Declarations

declaration $\rightarrow$ type identifier_list SEMICOLON
| type IDENTIFIER ASSIGN expression SEMICOLON

identifier_list $\rightarrow$ IDENTIFIER
| identifier_list COMMA IDENTIFIER

type $\rightarrow$ INT | FLOAT | BOOL | VOID

4.3 Functions

function $\rightarrow$ type IDENTIFIER LPAREN RPAREN LBRACE statement_list RBRACE

4.4 Statements

statement $\rightarrow$ declaration
| assignment_statement
| if_statement
| while_statement
| for_statement
| return_statement
| read_statement
| write_statement
| expression SEMICOLON

statement_list $\rightarrow$ statement_list statement
| statement
| /* empty */
| statement_list error SEMICOLON (error recovery)

assignment_statement $\rightarrow$ IDENTIFIER ASSIGN expression SEMICOLON

return_statement $\rightarrow$ RETURN expression SEMICOLON
| RETURN SEMICOLON

read_statement → READ LPAREN IDENTIFIER RPAREN SEMICOLON

write_statement → WRITE LPAREN expression RPAREN SEMICOLON

4.5 Control Structures

if_statement → IF LPAREN expression RPAREN if_block
 | IF LPAREN expression RPAREN if_block ELSE if_block

if_block → LBRACE statement_list RBRACE

while_statement → WHILE LPAREN expression RPAREN LBRACE statement_list RBRACE

for_statement → FOR LPAREN assignment SEMICOLON expression SEMICOLON assignment RPAREN

assignment → IDENTIFIER ASSIGN expression

4.6 Expressions

expression → expression OR expression
 | expression AND expression
 | expression EQ expression
 | expression NE expression
 | expression LT expression
 | expression LE expression
 | expression GT expression
 | expression GE expression
 | expression PLUS expression
 | expression MINUS expression
 | expression MULTIPLY expression
 | expression DIVIDE expression
 | expression MODULO expression
 | NOT expression
 | LPAREN expression RPAREN
 | IDENTIFIER
 | INT_LIT
 | FLOAT_LIT
 | BOOL_LIT

4.7 Operator Precedence

From lowest to highest:

1. OR (left-associative)
2. AND (left-associative)
3. EQ, NE (left-associative)
4. LT, LE, GT, GE (left-associative)
5. PLUS, MINUS (left-associative)
6. MULTIPLY, DIVIDE, MODULO (left-associative)
7. NOT (right-associative)

Note: The grammar has 68 shift/reduce conflicts, which are expected for expression grammars and are resolved using operator precedence.

5 Implementation Details

5.1 Lexical Analysis

Tool: Flex (Lex)

Key Features:

- Recognizes keywords, identifiers, literals, operators, and punctuation
- Handles whitespace and comments (`//` and `/* */`)
- Reports lexical errors for unexpected characters
- Tracks line numbers using `%option yylineno`

Tokens Recognized:

- Keywords: `int`, `float`, `bool`, `void`, `if`, `else`, `while`, `for`, `return`, `read`, `write`, `true`, `false`
- Identifiers: `[a-zA-Z][a-zA-Z0-9]*`
- Literals: Integer (`[0-9]+`), Float (`[0-9]+\.[0-9]+`), Boolean (`true`, `false`)
- Operators: Arithmetic, comparison, logical, assignment
- Punctuation: `;`, `,`, `(`, `)`, `{`, `}`, `[`, `]`

5.2 Syntax Analysis

Tool: Bison (Yacc)

Key Features:

- LALR(1) parser generated from grammar rules
- Operator precedence and associativity handled
- Error recovery using panic mode and error productions
- Integrates with semantic analysis, symbol table, and code generation during parsing

Error Recovery:

- Panic mode: Skips tokens until synchronization point (semicolon or brace)
- Error production: `statement list: statement list error SEMICOLON` recovers at statement boundaries
- Continues parsing after errors to detect multiple issues

5.3 Symbol Table Management

Data Structure: Hash table with chaining

Features:

- $O(1)$ average lookup time
- Scope management: Global (0), Function (1), Nested blocks (2+)
- Operations: `insert_symbol()`, `lookup_symbol()`, `enter_scope()`, `exit_scope()`
- Detects redeclarations in same scope
- Scope-aware lookup (searches current scope first, then outer scopes)

5.4 Semantic Analysis

Type Checking:

- Validates type compatibility in assignments (allows `int`→`float` conversion)
- Checks arithmetic operations (numeric types only)
- Validates logical operations (bool types only)
- Validates comparison operations (numeric and bool)

- Checks variable declarations before use

Supported Types:

- TYPE_INT - Integer
- TYPE_FLOAT - Floating-point
- TYPE_BOOL - Boolean
- TYPE_VOID - Void (for functions)
- TYPE_UNKNOWN - Error type

5.5 Error Handling

Error Types:

- Lexical errors (unexpected characters)
- Syntax errors (grammar violations)
- Semantic errors (type mismatches, undeclared variables, redeclarations)

Error Recovery:

- Continues parsing after errors
- Collects multiple errors before stopping
- Filters false positive errors that occur after recovery
- Displays post-recovery parsing results (symbols and intermediate code)

Error Reporting:

- Format: [Line N] ERROR_TYPE: message
- All errors displayed at end of compilation
- Shows error recovery indicator when applicable

5.6 Intermediate Code Generation

Format: Three-address code

Instruction Types:

- Arithmetic: ADD, SUB, MUL, DIV, MOD
- Comparison: LT, LE, GT, GE, EQ, NE
- Logical: AND, OR, NOT
- Assignment: ASSIGN
- Control Flow: GOTO, IF GOTO, LABEL
- Functions: FUNC.START, FUNC.END, RETURN
- I/O: READ, WRITE

Features:

- Generates temporary variables (t0, t1, t2, ...)
- Generates labels for control flow (L1, L2, L3, ...)
- Output written to output/intermediate_code.txt

6 Design Choices

6.1 Tool Selection

Flex for Lexical Analysis:

- Automatic token generation
- Efficient DFA construction
- Faster development than manual scanner

Bison for Syntax Analysis:

- LALR(1) parser generator
- Handles operator precedence automatically
- Automatic parser generation

6.2 Data Structures

Hash Table for Symbol Table:

- $O(1)$ average lookup time
- Efficient for large programs

Linked List for Error Collection:

- Dynamic size
- Easy insertion

Linked List for Intermediate Code:

- Dynamic code generation
- Sequential output

6.3 Error Recovery Strategy

Panic Mode Recovery:

- Simple and effective
- Continues parsing after errors
- Detects multiple errors in one pass
- Filters false positive errors

7 Testing

7.1 Test Cases

Valid Test Cases (5):

1. test valid 01.c - Basic declarations and assignments
2. test valid 02.c - Control structures (if-else, while, for)
3. test valid 03.c - Arithmetic and logical operations
4. test valid 04.c - Input/Output statements
5. test valid 05.c - Complex nested structures

Invalid Test Cases (6):

1. test_invalid 01.c - Undeclared variable
2. test_invalid 02.c - Type mismatch
3. test_invalid 03.c - Syntax error (missing semicolon)
4. test_invalid 04.c - Redeclaration
5. test_invalid 05.c - Invalid operation (arithmetic on bool)
6. test_invalid 06.c - Missing closing brace

Error Recovery Test Cases (10):

- test_recovery 01.c through test_recovery 10.c - Various error recovery scenarios

7.2 Test Results

Valid cases compile successfully and generate intermediate code

Invalid cases correctly detect and report errors

Error recovery test cases demonstrate continued parsing after errors

All compiler phases work correctly

8 Conclusion

This project successfully implements a mini-compiler for a C-like language subset, demonstrating:

Integration of all major compiler phases (Lexical, Syntax, Semantic, Code Generation)

Use of automated tools (Flex and Bison)

Comprehensive error handling and error recovery

Symbol table management with scope support

Intermediate code generation

Modular, extensible architecture

The compiler correctly processes valid programs and detects errors in invalid programs, meeting all project requirements. The implementation demonstrates understanding of compiler construction principles and system-level integration of compiler phases.